Microservicios y APIs Escalables

Universidad de Palermo · 2026

Sprint 13

Resiliencia: Circuit Breaker, Retry y Rate Limiter

Resilience4j · Spring Cloud Gateway

Objetivo del sprint

En este sprint vas a implementar resiliencia en la comunicación entre microservicios. Cuando un servicio falla o tarda demasiado, el sistema no debe caerse en cascada — debe degradar su comportamiento de forma controlada. Para eso usaremos **Resilience4j**, la librería de resiliencia estándar del ecosistema Spring Cloud.

Al finalizar el sprint, el sistema "Patitas Felices" va a:

- **Detectar** automáticamente cuándo un microservicio está fallando (Circuit Breaker)
- **Reintentar** llamadas fallidas con espera progresiva (Retry)
- **Limitar** la cantidad de llamadas por segundo para proteger los servicios (Rate Limiter)
- **Responder** con datos de fallback en vez de propagar errores al cliente

> 📌 **Contexto del sprint**
> En el Sprint 11 implementaste OpenFeign para que ms-turno llame a ms-mascota y ms-veterinario.
> En el Sprint 12 agregaste el API Gateway como único punto de entrada.
> En este sprint agregás la capa de resiliencia: ¿qué pasa si ms-mascota no responde?
> La respuesta correcta no es un error 500 — es un comportamiento de fallback controlado.

Marco conceptual

El problema: fallo en cascada

En una arquitectura de microservicios, si ms-mascota tarda 30 segundos en responder, **ms-turno queda bloqueado esperando** esa respuesta. Mientras tanto, ms-turno tampoco puede responder a sus clientes. El fallo se propaga como un dominó. Resilience4j implementa tres patrones para evitar esto:

Patrón	Problema que resuelve	Analogía
Circuit Breaker	Detecta servicios caídos y deja de	Disyuntor eléctrico: se abre

Patrón	Problema que resuelve	Analogía
	llamarlos temporalmente	cuando hay sobrecarga
Retry	Reintenta llamadas fallidas por errores transitorios	Volver a llamar al timbre si no atendieron la primera vez
Rate Limiter	Limita llamadas por segundo para proteger el servicio	Torniquete de subte: regula el flujo para no colapsar

Los tres estados del Circuit Breaker

Estado	Descripción	Qué hace el sistema
CLOSED (normal)	Llamadas pasan normalmente. Se cuentan éxitos y fallos.	Funciona como siempre. Acumula errores en ventana deslizante.
OPEN (abierto)	Superó el umbral de fallos. Las llamadas se bloquean.	Devuelve el fallback directamente. No molesta al servicio caído.
HALF_OPEN (semiabierto)	Después del tiempo de espera, prueba con pocas llamadas.	Si tienen éxito → CLOSED. Si fallan → vuelve a OPEN.

📌 **Parámetros clave del Circuit Breaker**

slidingWindowSize: tamaño de la ventana para calcular la tasa de error (ej: últimas 10 llamadas)
failureRateThreshold: porcentaje de fallos para abrir el circuito (ej: 50% = 5 de 10)
waitDurationInOpenState: tiempo que espera en OPEN antes de probar en HALF_OPEN (ej: 10s)
permittedNumberOfCallsInHalfOpenState: llamadas de prueba en HALF_OPEN (ej: 3)

PASO 1 — Agregar dependencias de Resilience4j en ms-turno

Módulo: ms-turno (pom.xml)

Agregá las siguientes dependencias en el pom.xml de ms-turno:

Primera dependencia — Resilience4j para Spring Boot:

```
<!-- Resilience4j -->
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
```

Segunda dependencia — Spring AOP (requerida para las anotaciones):

```
<!-- AOP para que funcionen @CircuitBreaker y @Retry -->
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

> ⚠️ **Dependencia AOP es obligatoria**
> Sin spring-boot-starter-aop, las anotaciones @CircuitBreaker y @Retry no tienen efecto.
> El código compila pero el comportamiento de resiliencia no se activa.
> Es un error silencioso — no hay excepción, simplemente no funciona.

La versión de resilience4j-spring-boot3 es administrada por el BOM de Spring Cloud (`spring-cloud-dependencies 2025.0.0`). No especificar versión manualmente.

PASO 2 — Configurar Circuit Breaker y Retry en application.yml

Archivo: ms-turno/src/main/resources/application.yml

Agregá la configuración de Resilience4j al final del application.yml de ms-turno. Definí dos instancias de Circuit Breaker (una para mascotaService y otra para veterinarioService) y dos instancias de Retry:

Sección resilience4j — Circuit Breaker:

```
resilience4j:
  circuitbreaker:
    instances:
      mascotaService:
        registerHealthIndicator: true
        slidingWindowSize: 10
        minimumNumberOfCalls: 5
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
        permittedNumberOfCallsInHalfOpenState: 3
        slidingWindowType: COUNT_BASED
      veterinarioService:
        registerHealthIndicator: true
        slidingWindowSize: 10
        minimumNumberOfCalls: 5
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
        permittedNumberOfCallsInHalfOpenState: 3
        slidingWindowType: COUNT_BASED
```

Sección resilience4j — Retry:

```
  retry:
    instances:
      mascotaService:
        maxAttempts: 3
        waitDuration: 1s
```

```
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
      veterinarioService:
        maxAttempts: 3
        waitDuration: 1s
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
```

📌 **Explicación de los parámetros de Retry**

maxAttempts: 3 → intenta 3 veces antes de rendirse (incluye el intento original)
waitDuration: 1s → espera 1 segundo entre intentos
enableExponentialBackoff: true → en vez de esperar siempre 1s, va 1s → 2s → 4s
exponentialBackoffMultiplier: 2 → multiplica por 2 en cada intento

PASO 3 — Crear ServicioNoDisponibleException

Contexto: ms-turno · paquete domain/exception

Creá la excepción `ServicioNoDisponibleException` en el paquete domain/exception de ms-turno. Esta excepción se lanza en los métodos de fallback cuando el circuito está abierto o los reintentos se agotaron:

```
// Estructura del archivo:
package com.patitas.msturno.domain.exception;

// Clase: ServicioNoDisponibleException
// Extiende: RuntimeException
// Constructor: recibe String mensaje
// Llama a super(mensaje)
```

Esta excepción debe ser manejada en el GlobalExceptionHandler de ms-turno con una respuesta HTTP 503 (Service Unavailable).

Actualizá el GlobalExceptionHandler de ms-turno para capturar `ServicioNoDisponibleException` y responder con:

- **HTTP Status:** 503 SERVICE_UNAVAILABLE
- **Body:** mensaje descriptivo indicando qué servicio no está disponible

PASO 4 — Agregar @CircuitBreaker y @Retry en MascotaFeignAdapter

Archivo: MascotaFeignAdapter.java (adaptador de salida en ms-turno)

Este adaptador ya existe del Sprint 11. Ahora le agregás las anotaciones de resiliencia en su método principal. El método que llama a ms-mascota (por ejemplo `obtenerMascota(Long id)`) recibe dos anotaciones nuevas:

Anotación 1 — Circuit Breaker:

```
@CircuitBreaker(name = "mascotaService", fallbackMethod = "fallbackMascota")
```

Anotación 2 — Retry:

```
@Retry(name = "mascotaService", fallbackMethod = "fallbackMascota")
```

El método `fallbackMascota` debe existir en la misma clase. Recibe los mismos parámetros que el método original, más un parámetro extra `Throwable ex` al final:

```
// Firma del método fallback:
// - Mismo tipo de retorno que el método principal
// - Mismos parámetros del método principal
// - + Throwable ex al final
// - Visibilidad: private

private MascotaResponse fallbackMascota(Long mascotaId, Throwable ex) {
    throw new ServicioNoDisponibleException(
        "El servicio de mascotas no está disponible: " + ex.getMessage()
    );
}
```

⚠️ **Reglas del método fallback**

1. El nombre debe coincidir exactamente con fallbackMethod="..."
2. El tipo de retorno debe ser idéntico al método principal.
3. Los parámetros deben ser los mismos + Throwable al final.
4. Si no coincide la firma, Resilience4j lanza una excepción en runtime.

PASO 5 — Agregar @CircuitBreaker y @Retry en VeterinarioFeignAdapter

Archivo: VeterinarioFeignAdapter.java (adaptador de salida en ms-turno)

Repetí el mismo proceso del Paso 4 para el adaptador de veterinario. El método que llama a ms-veterinario recibe las mismas anotaciones, con el nombre `"veterinarioService"` (coincide con la instancia configurada en el YAML):

Anotaciones:

```
@CircuitBreaker(name = "veterinarioService", fallbackMethod = "fallbackVeterinario")
@Retry(name = "veterinarioService", fallbackMethod = "fallbackVeterinario")
```

Método fallback:

```
private VeterinarioResponse fallbackVeterinario(Long veterinarioId, Throwable ex) {
    throw new ServicioNoDisponibleException(
        "El servicio de veterinarios no está disponible: " + ex.getMessage()
    );
}
```

📌 **Por qué el name debe coincidir con el YAML**

El name="mascotaService" en la anotación debe coincidir exactamente con el nombre de la instancia definida en resilience4j.circuitbreaker.instances en el YAML.
Si no coincide, Resilience4j usa los valores por defecto (no los que configuraste).
Es un error silencioso: el código funciona pero con configuración incorrecta.

PASO 6 — Configurar Rate Limiter en el API Gateway

Módulo: api-gateway · application.yml

El Rate Limiter se configura en el Gateway para proteger a los microservicios de un volumen excesivo de peticiones. Cuando se supera el límite, el gateway responde **HTTP 429 Too Many Requests** sin llegar a llamar al microservicio.

Primero: dependencia en pom.xml del api-gateway:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

Segundo: configuración del filtro RequestRateLimiter en la ruta de ms-turno:

```
spring:
  cloud:
    gateway:
      routes:
        - id: ms-turno
          uri: lb://ms-turno
          predicates:
            - Path=/api/turnos/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
                redis-rate-limiter.requestedTokens: 1
```

📌 **Explicación del Rate Limiter (algoritmo Token Bucket)**

replenishRate: 10 → se generan 10 tokens por segundo (10 llamadas/seg normales)
burstCapacity: 20 → el balde puede tener máximo 20 tokens (picos de hasta 20)
requestedTokens: 1 → cada llamada consume 1 token

Si el balde está vacío → HTTP 429. Los tokens se regeneran aunque haya 429s.

⚠️ **Redis es obligatorio para RequestRateLimiter**

Spring Cloud Gateway usa Redis para almacenar el estado del rate limiter.
Sin Redis corriendo, el gateway no arranca con esta configuración.

> Para desarrollo local, levantá Redis con Docker:
> docker run -d -p 6379:6379 redis:alpine

PASO 7 — Exponer Circuit Breaker en Actuator

Archivo: ms-turno/src/main/resources/application.yml

Resilience4j se integra con Spring Boot Actuator y expone endpoints para monitorear el estado de los circuit breakers en tiempo real. Actualizá la configuración de Actuator en ms-turno:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,circuitbreakers,circuitbreakerevents
  endpoint:
    health:
      show-details: always
  health:
    circuitbreakers:
      enabled: true
```

Endpoints disponibles después de esta configuración:

Endpoint	Qué muestra
GET /actuator/health	Estado general + estado de cada circuit breaker (CLOSED/OPEN/HALF_OPEN)
GET /actuator/circuitbreakers	Lista de todos los circuit breakers con sus métricas
GET /actuator/circuitbreakerevents	Historial de eventos: llamadas exitosas, fallos, transiciones de estado
GET /actuator/metrics/resilience4j.circuitbreaker.state	Métrica puntual del estado actual

PASO 8 — Verificación end-to-end

Verificá que todo funciona correctamente con tres escenarios de prueba:

Escenario 1 — Flujo normal (circuito CLOSED):

- Con todos los microservicios corriendo, realizá una petición de creación de turno vía el Gateway
- El turno debe crearse correctamente con datos del dueño, mascota y veterinario
- Verificá en GET /actuator/health que los circuit breakers muestran estado CLOSED

Escenario 2 — Fallback activo (circuito OPEN):

- Detenné ms-mascota (cerrá el proceso o el contenedor)
- Realizá varias peticiones de turno (más de 5, que es el minimumNumberOfCalls configurado)
- Las primeras peticiones devuelven 503 con mensaje de fallback
- Después de superar el failureRateThreshold (50%), el circuito pasa a OPEN
- Verificá en /actuator/health que mascotaService muestra OPEN
- Con el circuito OPEN, las respuestas son inmediatas (no espera timeout)

Escenario 3 — Recuperación (HALF_OPEN → CLOSED):

- Después de 10 segundos (waitDurationInOpenState), el circuito pasa a HALF_OPEN
- Volvé a levantar ms-mascota
- Las siguientes 3 llamadas (permittedNumberOfCallsInHalfOpenState) son de prueba
- Si tienen éxito, el circuito vuelve a CLOSED
- Verificá la transición en /actuator/circuitbreakerevents

> 📌 **Escenario extra: Rate Limiter (opcional)**
> Usá Apache Bench (ab) o Postman Runner para enviar más de 20 peticiones/segundo al endpoint /api/turnos del Gateway.
> Las peticiones que excedan el límite deben recibir HTTP 429 Too Many Requests.
> Las peticiones dentro del límite siguen procesándose normalmente.

Resumen del sprint

Paso	Qué hiciste	Dónde
1	Dependencias Resilience4j + AOP	ms-turno / pom.xml
2	Config Circuit Breaker + Retry en YAML	ms-turno / application.yml
3	ServicioNoDisponibleException + GlobalExceptionHandler 503	ms-turno / domain/exception
4	@CircuitBreaker + @Retry en MascotaFeignAdapter	ms-turno / adapters/out
5	@CircuitBreaker + @Retry en VeterinarioFeignAdapter	ms-turno / adapters/out
6	Rate Limiter (RequestRateLimiter + Redis) en Gateway	api-gateway / application.yml
7	Actuator endpoints de Circuit Breaker	ms-turno / application.yml
8	Verificación: 3 escenarios end-to-end	Postman / curl

📌 **Reglas de este sprint**

- El name en @CircuitBreaker y @Retry debe coincidir con la instancia en el YAML.
- El fallbackMethod debe tener los mismos parámetros + Throwable al final.
- spring-boot-starter-aop es obligatorio para que las anotaciones funcionen.
- El Rate Limiter en el Gateway requiere Redis. Levantarlo con Docker para testing.
- Resilience4j se aplica en los adaptadores de salida (Feign), no en los controllers.